

手摸手之实现敏感信息加密存储

需求背景

安全控制一直是治理的重要环节，数据加密属于安全控制的范畴。无论对互联网公司还是传统行业来说，数据安全一直是极为重视和敏感的话题。数据加密是指对某些敏感信息通过加密规则进行数据的变形，实现敏感隐私数据的可靠保护。涉及客户安全数据或者一些商业性敏感数据，如身份证号、手机号、卡号、客户号等个人信息按照相关部门规定，都需要进行数据加密。

对于数据加密的需求，在现实的业务场景中存在如下情况：

- 安全部门规定需将涉及用户敏感信息，例如银行、手机号码等进行加密后存储到数据库，在使用的时候再进行解密处理。

在真实业务场景中，相关业务开发团队则往往需要针对公司安全部门需求，自行实行并维护一套加解密系统。而当加密场景发生改变时，自行维护的加密系统往往又面临着重构或修改风险。此外，对于已经上线的业务，在不修改业务逻辑和 SQL 的情况下，透明化、安全低风险地实现无缝进行加密改造也相对复杂。

项目实战

我们以用户表举例，用户表中存在证件号、手机号、邮箱以及地址等敏感字段，需要在数据库中进行加密存储。

代码实战查看 `/services/user-service`。

引入 ShardingSphere

```
<dependency>
  <groupId>org.apache.shardingsphere</groupId>
  <artifactId>shardingsphere-jdbc-core</artifactId>
  <version>5.3.2</version>
</dependency>
```

新增加密配置

`application.yaml` 配置文件修改配置，将数据库驱动变更为 ShardingSphere Driver。

```
spring:
  datasource:
    driver-class-name: org.apache.shardingsphere.driver.ShardingSphereDriver
    url: jdbc:shardingsphere:classpath:shardingsphere-config.yaml
```

并配置 `shardingsphere-config.yaml` 相关配置。为了方便大家理解，我把分库分表的相关配置删除了，仅保留了加密相关的配置。

```
# 配置数据源，底层被 ShardingSphere 进行了代理
datasources:
  ds_0:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: com.mysql.cj.jdbc.Driver
    jdbcUrl: jdbc:mysql://127.0.0.1:3306/12306_user_0?
    useUnicode=true&characterEncoding=UTF-
    8&rewriteBatchedStatements=true&allowMultiQueries=true&serverTimezone=Asia/Shanghai
    username: root
    password: root

rules:
# 数据加密存储规则
- !ENCRYPT
# 需要加密的表集合
tables:
# 用户表
t_user:
# 用户表中哪些字段需要进行加密
columns:
# 身份证字段，逻辑字段，不一定是在数据库中真实存在
id_card:
# 身份证字段存储的密文字段，这个是数据库中真实存在的字段
cipherColumn: id_card
# 身份证字段加密算法
encryptorName: common_encryptor
phone:
cipherColumn: phone
```

```

        encryptorName: common_encryptor
    mail:
        cipherColumn: mail
        encryptorName: common_encryptor
    address:
        cipherColumn: address
        encryptorName: common_encryptor
    # 是否按照密文字段查询
    queryWithCipherColumn: true
# 加密算法
encryptors:
    # 自定义加密算法名称
    common_encryptor:
        # 加密算法类型
        type: AES
        props:
            # AES 加密密钥
            aes-key-value: d6oadClrrb9A3Gwo
props:
    sql-show: true

```

效果展示

咱们新插入一条用户信息，在应用程序里还是明文，经过 ShardingSphere 代理后，存储数据库时，就已经是密文的了。

原始 SQL:

```

insert into t_user (id_card, phone, mail, address) values ('34020xx023081xx338',
'1x60111xx983', 'mading@axxche.org', 'xx东城x');

```

修改后的 SQL:

```

insert into t_user (id_card, phone, mail, address) values
('YUvr+8Xf17VCgGonU2WXqmKuhB5FMazUEbh3y+h0B38=', 'MZObk+5TeYPLHtP2A6+aIW==',
'vX/5iWTyfAvMJMt+ioipj9vd6cnZ4rz4qKBAXQ9C9oU=',
'vX/5iWTyfAvMJMt+ioipj9vd6cnZ4rz4qKBAXQ9C9oU=');

```

可能有小伙伴比较疑问，存储到数据库是密文，那查询时展示不就有问题了吗？

ShardingSphere 在执行查询语句时，如果涉及到相关加密表，会自动将加密数据转换为明文数据，也就是会把 YUvr+8Xf17VCgGonU2WXqmKuhB5FMazUEbh3y+h0B38= 转换为

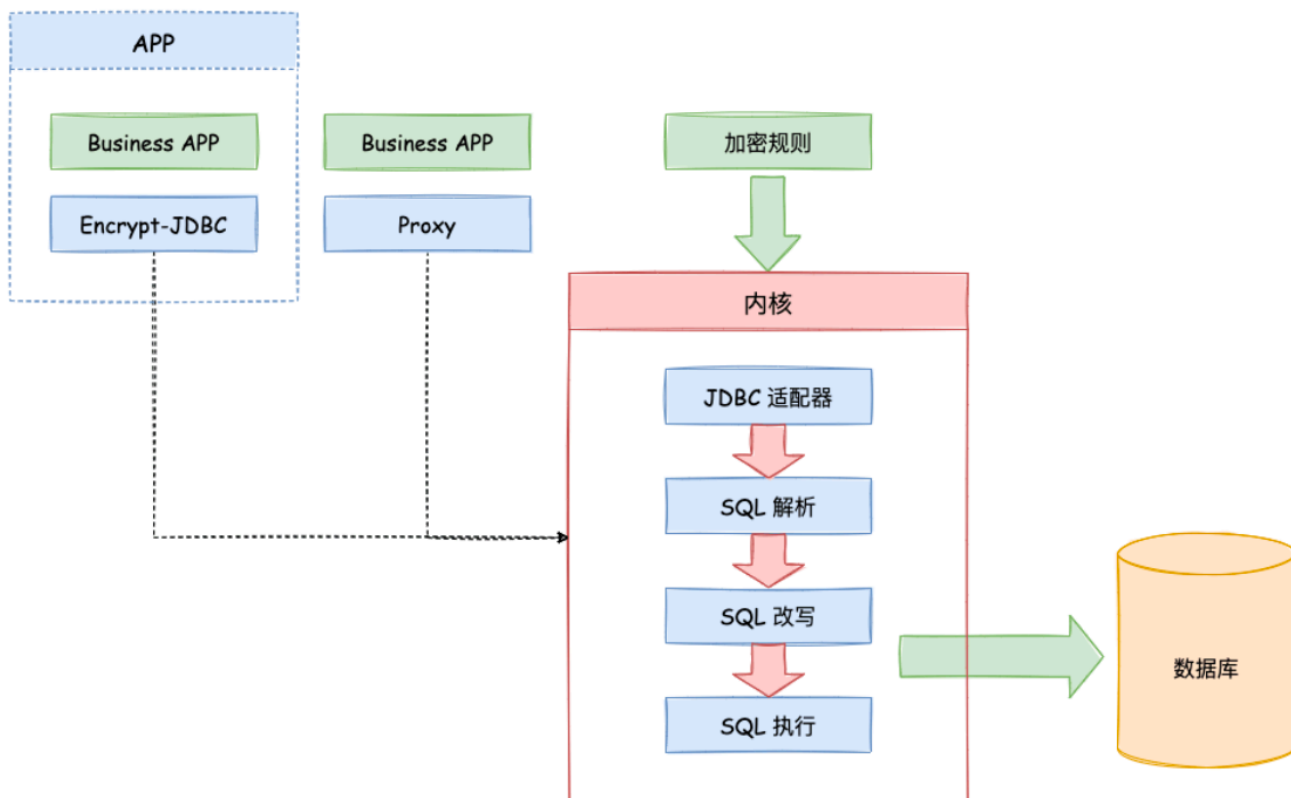
34020xx023081xx338。

这样就能形成一个加密敏感信息落库闭环。

实现原理

加密模块将用户发起的 SQL 进行拦截，并通过 SQL 语法解析器进行解析、理解 SQL 行为，再依据用户传入的加密规则，找出需要加密的字段和所使用的加解密算法对目标字段进行加解密处理后，再与底层数据库进行交互。

Apache ShardingSphere 会将用户请求的明文进行加密后存储到底层数据库；并在用户查询时，将密文从数据库中取出进行解密后返回给终端用户。通过屏蔽对数据的加密处理，使用户无需感知解析 SQL、数据加密、数据解密的处理过程，就像在使用普通数据一样使用加密数据。



加密功能扩展

如果大家理解了上面说的加密逻辑，我们假设有个新上线的业务，使用这种模型是完全没有问题的。但是，我们再深度思考个问题，如果说数据库表中已经有这种明文的敏感信息了，如何解决？

首先需要对这个敏感信息清洗成密文的数据，其次为了保障业务可能，需要是不停机清洗的那种。

参考链接：

[如何防止用户敏感数据泄露](#)

[数据加密 :: ShardingSphere](#)

[数据加密 :: ShardingSphere](#)

[数据加密 :: ShardingSphere](#)

更新: 2023-08-12 19:36:21

原文: <<https://www.yuque.com/magestack/12306/vhf4i3c604t2qex7>>